

C++ Memory Management

1. Overview

C++ memory management is about how a program gets memory, uses it safely, and releases it at the right time.

Definition: In C++, memory can be stored in different areas such as the stack, heap, static storage, and read-only program sections. The programmer often controls object lifetime directly, especially when using `new`, `delete`, `malloc`, `free`, pointers, and resource-owning classes.

Why it matters: Bad memory management can cause crashes, memory leaks, dangling pointers, double frees, corrupted data, security bugs, and unpredictable behavior.

Where it is used in real systems:

- Operating systems: process memory, kernel buffers, device drivers.
- Databases: page cache, query execution buffers, indexes.
- Browsers: DOM objects, JavaScript engines, rendering pipelines.
- Backend servers: request objects, connection pools, caches.
- Games and embedded systems: strict memory and performance control.

Why interviewers ask about it: Memory management tests whether you understand object lifetime, ownership, copying, performance, and undefined behavior. These are core C++ skills and common causes of production bugs.

2. Core Idea

The core idea is simple: every object has a lifetime, and every dynamically acquired resource must have a clear owner responsible for releasing it.

Intuition

Think of memory like rooms in a building:

- Stack memory is like a temporary meeting room automatically cleaned when the meeting ends.
- Heap memory is like rented storage space. You must explicitly return it, or use a manager that returns it for you.
- Smart pointers are like rental contracts that clearly say who owns the storage and when it should be released.
- RAII is the rule: tie the resource to an object, so cleanup happens automatically when the object dies.

Small Example

```
#include <memory>
#include <iostream>

void example() {
```

```
int stackValue = 10;           // automatic lifetime
int* raw = new int(20);        // heap allocation
auto smart = std::make_unique<int>(30); // heap allocation with ownership

std::cout << stackValue << " " << *raw << " " << *smart << "\n";

delete raw;                    // manual cleanup required
}                               // smart is automatically deleted
here
```

Step-by-Step Explanation

1. `stackValue` is created on the stack when `example()` is called.
2. `raw` points to an `int` allocated on the heap using `new`.
3. `smart` owns a heap object through `std::unique_ptr`.
4. `delete raw` releases the manually allocated heap memory.
5. When the function ends, stack variables are destroyed automatically.
6. `smart` automatically calls `delete` for its owned object.

The interview lesson: modern C++ prefers automatic lifetime and smart pointers over manual `new` and `delete`.

3. Important Subtopics

3.1 Stack Memory

What it means: Stack memory stores local variables, function parameters, return addresses, and temporary objects with automatic lifetime.

Why it matters: Stack allocation is fast and automatically cleaned up, but stack size is limited.

Example:

```
void solve() {
    int x = 5;
    int arr[100];
}
```

`x` and `arr` are destroyed when `solve()` returns.

Common interview angle: Interviewers ask whether returning the address of a local variable is safe.

```
int* bad() {
    int x = 10;
    return &x; // wrong: x dies when function returns
}
```

3.2 Heap Memory

What it means: Heap memory is dynamically allocated memory that can live beyond the current function call.

Why it matters: It supports flexible lifetimes and large objects, but manual management is dangerous.

Example:

```
int* p = new int(42);
delete p;
```

Common interview angle: Explain why every successful `new` should be matched with `delete`, unless ownership is transferred to a smart pointer or container.

3.3 `new` and `delete`

What it means: `new` allocates memory and constructs an object. `delete` destroys the object and releases memory.

Why it matters: `new/delete` understand C++ constructors and destructors.

Example:

```
class Student {
public:
    Student() { std::cout << "constructed\n"; }
    ~Student() { std::cout << "destroyed\n"; }
};

Student* s = new Student();
delete s;
```

Common interview angle: Difference between `new/delete` and `malloc/free`.

3.4 `new[]` and `delete[]`

What it means: `new[]` allocates an array of objects. `delete[]` destroys all elements and releases the array memory.

Why it matters: Mixing `new[]` with `delete` is undefined behavior.

Example:

```
int* arr = new int[5];
delete[] arr;
```

Common interview angle: What happens if you write `delete arr;` instead of `delete[] arr;`? Answer: undefined behavior.

3.5 `malloc` and `free`

What it means: `malloc` allocates raw memory. `free` releases raw memory. They come from C, not modern C++ style.

Why it matters: `malloc` does not call constructors, and `free` does not call destructors.

Example:

```
#include <cstdlib>

int* p = static_cast<int*>(std::malloc(sizeof(int)));
*p = 10;
std::free(p);
```

Common interview angle: `malloc/free` should not be mixed with `new/delete`.

3.6 Memory Leaks

What it means: A memory leak happens when allocated memory is no longer reachable or no longer freed.

Why it matters: Leaks increase memory usage and can crash long-running systems.

Example:

```
void leak() {
    int* p = new int(10);
    // delete p is missing
}
```

Common interview angle: Leaks are especially serious in servers, daemons, browsers, and games because they run for a long time.

3.7 Dangling Pointers

What it means: A dangling pointer points to memory that has already been destroyed or freed.

Why it matters: Dereferencing it causes undefined behavior.

Example:

```
int* p = new int(5);
delete p;
std::cout << *p; // wrong: p is dangling
```

Common interview angle: Setting `p = nullptr` after `delete` avoids accidental reuse of that pointer, but it does not fix other aliases pointing to the same memory.

3.8 Smart Pointers

What it means: Smart pointers are RAII classes that manage dynamic memory automatically.

Why it matters: They make ownership explicit and reduce leaks.

Example:

```
#include <memory>

auto p = std::make_unique<int>(10);
auto q = std::make_shared<int>(20);
```

Common interview angle: Choose `unique_ptr` for exclusive ownership, `shared_ptr` for shared ownership, and `weak_ptr` to break shared ownership cycles.

3.9 RAII

What it means: RAII stands for Resource Acquisition Is Initialization. Acquire a resource in a constructor and release it in a destructor.

Why it matters: Cleanup happens automatically even during exceptions.

Example:

```
#include <fstream>

void writeFile() {
    std::ofstream file("out.txt");
    file << "hello";
} // file closes automatically
```

Common interview angle: RAII is not only for memory. It is also used for files, locks, sockets, database connections, and transactions.

3.10 Shallow Copy

What it means: A shallow copy copies pointer values, not the data being pointed to.

Why it matters: Two objects may accidentally own the same resource, causing double delete or shared mutation bugs.

Example:

```
class Buffer {
public:
    int* data;
};

Buffer a;
a.data = new int(5);
Buffer b = a; // shallow copy: both point to same int
```

Common interview angle: If both destructors call `delete data`, the program may double free.

3.11 Deep Copy

What it means: A deep copy creates a new resource and copies the content into it.

Why it matters: Each object owns its own independent copy.

Example:

```
class Buffer {
    int* data;

public:
    Buffer(int value) : data(new int(value)) {}

    Buffer(const Buffer& other) : data(new int(*other.data)) {}

    ~Buffer() {
        delete data;
    }
};
```

Common interview angle: If a class owns raw memory, it usually needs a custom copy constructor, copy assignment operator, and destructor.

3.12 Rule of 3

What it means: If a class needs any one of these, it probably needs all three:

- Destructor
- Copy constructor
- Copy assignment operator

Why it matters: These functions control ownership and copying of resources.

Example:

```
class Buffer {
    int* data;

public:
    Buffer(int value) : data(new int(value)) {}
    ~Buffer() { delete data; }

    Buffer(const Buffer& other) : data(new int(*other.data)) {}

    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
            int* newData = new int(*other.data);
            delete data;
            data = newData;
        }
        return *this;
    }
};
```

Common interview angle: Self-assignment and exception safety in copy assignment.

3.13 Rule of 5

What it means: In C++11 and later, if a class manages a resource, also consider move operations:

- Destructor
- Copy constructor
- Copy assignment operator
- Move constructor
- Move assignment operator

Why it matters: Move operations avoid expensive deep copies and enable efficient transfers of ownership.

Example:

```
class Buffer {
    int* data;

public:
    Buffer(int value) : data(new int(value)) {}
    ~Buffer() { delete data; }

    Buffer(const Buffer& other) : data(new int(*other.data)) {}

    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
            int* newData = new int(*other.data);
            delete data;
            data = newData;
        }
    }
};
```

```
        return *this;
    }

    Buffer(Buffer&& other) noexcept : data(other.data) {
        other.data = nullptr;
    }

    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete data;
            data = other.data;
            other.data = nullptr;
        }
        return *this;
    }
};
```

Common interview angle: Move constructor transfers ownership and leaves the moved-from object valid but unspecified.

3.14 Rule of 0

What it means: Prefer classes that do not manually manage resources. Use standard library types that already handle memory.

Why it matters: It reduces bugs and lets the compiler generate correct special member functions.

Example:

```
#include <vector>

class Buffer {
    std::vector<int> data;
};
```

Common interview angle: Best modern C++ answer: avoid raw owning pointers; use RAII containers and smart pointers.

4. Real-World Example

Backend Server Request Handling

Imagine a C++ backend server handling thousands of HTTP requests.

For each request:

1. A thread receives request data.
2. Temporary variables are stored on the stack.
3. Large request bodies may be stored in heap-backed buffers like `std::vector<char>`.
4. Database connection handles are wrapped in RAII objects.

5. Locks are managed by `std::lock_guard` or `std::unique_lock`.
6. Response objects are destroyed automatically when request handling finishes.

Bad memory management in this system can cause:

- Leaks after every request, slowly increasing memory usage.
- Dangling pointers if cached objects are deleted too early.
- Double frees when ownership is unclear.
- Deadlocks if locks are not released during exceptions.

Modern C++ solves most of this with RAI:

```
void handleRequest() {
    std::vector<char> buffer(4096);
    std::unique_ptr<Request> req = std::make_unique<Request>();
    std::lock_guard<std::mutex> lock(globalMutex);

    process(*req, buffer);
} // buffer, req, and lock are cleaned up automatically
```

5. Diagrams / Mental Models

Process Memory Layout

```
High Address
+-----+
| Stack          |
| local variables, calls |
| grows downward |
+-----+
| Free space     |
+-----+
| Heap           |
| dynamic allocations |
| grows upward   |
+-----+
| Global / Static storage |
+-----+
| Code / Text segment |
+-----+
Low Address
```

Ownership Mental Model

```
Raw pointer:
int* p
"I point to something, but ownership is unclear."
```

```
unique_ptr:
    std::unique_ptr<T> p
    "I am the only owner."

shared_ptr:
    std::shared_ptr<T> p
    "Ownership is shared using reference counting."

weak_ptr:
    std::weak_ptr<T> p
    "I observe a shared object without owning it."
```

Manual Allocation Flow

```
new T
|
v
allocate memory
|
v
call constructor
|
v
use object
|
v
delete pointer
|
v
call destructor
|
v
release memory
```

RAII Flow

```
Object created
|
v
Constructor acquires resource
|
v
Object is used
|
v
Scope ends, return happens, or exception occurs
|
```

v

Destructor releases resource automatically

6. Common Interview Questions

Q1. What is the difference between stack and heap memory?

Answer: Stack memory is automatically managed and used for local variables and function calls. Heap memory is dynamically allocated and must be managed manually or through RAII objects like containers and smart pointers.

Expected key points: lifetime, allocation speed, size limits, automatic vs dynamic management.

Common mistakes: Saying stack is always tiny and heap is always huge without mentioning implementation details; saying all objects created with constructors are heap objects.

Q2. What does `new` do in C++?

Answer: `new` allocates memory and calls the constructor for the object.

Expected key points: allocation plus construction; returns typed pointer; throws `std::bad_alloc` on failure by default.

Common mistakes: Saying `new` only allocates memory; confusing it with `malloc`.

Q3. What does `delete` do?

Answer: `delete` calls the destructor and releases memory allocated by `new`.

Expected key points: destruction plus deallocation; pointer must come from compatible `new`; deleting `nullptr` is safe.

Common mistakes: Calling `delete` twice; using `delete` on stack memory.

Q4. Difference between `malloc/free` and `new/delete`?

Answer: `malloc/free` allocate and release raw memory. They do not call constructors or destructors. `new/delete` are C++ operators that allocate/deallocate memory and construct/destroy objects.

Expected key points: constructors, destructors, return type, error behavior, mixing is invalid.

Common mistakes: Using `free` on memory allocated by `new`.

Q5. What is a memory leak?

Answer: A memory leak occurs when allocated memory is not released and the program loses the ability or discipline to free it.

Expected key points: unreachable or unreleased memory; long-running process impact.

Common mistakes: Thinking leaks only happen when a program crashes; ignoring leaks hidden behind overwritten pointers.

Q6. What is a dangling pointer?

Answer: A dangling pointer points to memory that is no longer valid because the object was destroyed or memory was freed.

Expected key points: use-after-free, returning address of local variable, undefined behavior.

Common mistakes: Thinking setting one pointer to `nullptr` fixes all aliases.

Q7. What is RAII?

Answer: RAII is a C++ pattern where a resource is acquired in an object's constructor and released in its destructor.

Expected key points: deterministic cleanup, exception safety, resources beyond memory.

Common mistakes: Saying RAII is only about heap memory.

Q8. What are smart pointers?

Answer: Smart pointers are objects that manage raw pointers using RAII. They automatically release memory when ownership ends.

Expected key points: `unique_ptr`, `shared_ptr`, `weak_ptr`, ownership semantics.

Common mistakes: Using `shared_ptr` everywhere; forgetting cycles with `shared_ptr`.

Q9. When should you use `unique_ptr`?

Answer: Use `unique_ptr` when exactly one object owns a heap resource.

Expected key points: exclusive ownership, movable but not copyable, preferred default for owning pointers.

Common mistakes: Returning raw owning pointers instead of `unique_ptr`.

Q10. When should you use `shared_ptr`?

Answer: Use `shared_ptr` when multiple owners must share responsibility for an object's lifetime.

Expected key points: reference counting, overhead, risk of cycles, use only when shared ownership is real.

Common mistakes: Treating `shared_ptr` as a general replacement for all pointers.

Q11. What is shallow copy vs deep copy?

Answer: A shallow copy copies pointer values, so two objects may point to the same resource. A deep copy allocates a new resource and copies the content.

Expected key points: ownership, aliasing, double deletion risk.

Common mistakes: Thinking every compiler-generated copy is deep.

Q12. What is the Rule of 3?

Answer: If a class needs a destructor, copy constructor, or copy assignment operator, it probably needs all three.

Expected key points: raw resource ownership, copy behavior, cleanup behavior.

Common mistakes: Writing only a destructor for a raw owning pointer class.

Q13. What is the Rule of 5?

Answer: The Rule of 5 extends the Rule of 3 by adding move constructor and move assignment operator.

Expected key points: C++11 move semantics, efficient ownership transfer.

Common mistakes: Forgetting to set the moved-from object's pointer to `nullptr`.

Q14. What is the Rule of 0?

Answer: Design classes so they do not manually manage resources. Use standard containers, smart pointers, and RAII types so compiler-generated special members are correct.

Expected key points: modern C++ preference, fewer bugs, easier maintenance.

Common mistakes: Believing every class should manually implement copy and move operations.

Q15. Is it safe to call `delete` on `nullptr`?

Answer: Yes. Deleting `nullptr` has no effect.

Expected key points: safe no-op; still avoid unnecessary deletes in modern C++.

Common mistakes: Writing extra checks like `if (p) delete p;` as if required.

7. Deep-Dive Questions

Q1. Why is mixing `new/delete` with `malloc/free` undefined behavior?

`new` and `delete` are a matched pair. `new` may store implementation-specific allocation metadata and calls constructors. `delete` calls destructors and uses the matching deallocation function. `malloc/free` only deal with raw memory. Mixing them breaks object lifetime and allocator expectations.

Q2. Why can `shared_ptr` cause memory leaks?

`shared_ptr` uses reference counting. If two objects own each other through `shared_ptr`, their reference counts never become zero.

```
struct B;

struct A {
    std::shared_ptr<B> b;
};

struct B {
```

```
std::shared_ptr<A> a;
};
```

Use `std::weak_ptr` for one side of the relationship.

Q3. What does "valid but unspecified state" mean after move?

After moving from an object, it must remain destructible and assignable, but its exact value is not guaranteed unless the type documents it. You should not depend on its previous content.

```
std::string a = "hello";
std::string b = std::move(a);
// a is valid, but its content is unspecified
```

Q4. What is exception safety in copy assignment?

If allocation fails during copy assignment, the original object should ideally remain unchanged. A safer pattern is to allocate the new resource first, then release the old resource.

```
Buffer& operator=(const Buffer& other) {
    if (this != &other) {
        int* newData = new int(*other.data);
        delete data;
        data = newData;
    }
    return *this;
}
```

This avoids losing the original data if `new` throws.

Q5. Why should destructors usually not throw exceptions?

Destructors are often called during stack unwinding after another exception. If a destructor throws during this process, the program may call `std::terminate`. Destructors should usually catch errors internally or provide separate explicit cleanup functions if failure must be reported.

8. Comparison Tables

Stack vs Heap

Feature	Stack	Heap
Lifetime	Automatic scope-based lifetime	Programmer or owner-controlled lifetime
Speed	Usually very fast	Usually slower than stack
Size	Limited	Larger, but still finite

Feature	Stack	Heap
Used for	Local variables, function calls	Dynamic objects, large/flexible data
Cleanup	Automatic	Manual or RAII-based
Common bug	Returning pointer/reference to local variable	Leak, dangling pointer, double free

new/delete vs malloc/free

Feature	new/delete	malloc/free
Language	C++	C
Constructor called	Yes, by <code>new</code>	No
Destructor called	Yes, by <code>delete</code>	No
Return type	Typed pointer	<code>void*</code>
Failure behavior	Throws <code>std::bad_alloc</code> by default	Returns <code>NULL</code>
Recommended in modern C++	Rarely directly	Rarely in C++ application code

unique_ptr vs shared_ptr vs weak_ptr

Smart pointer	Ownership	Copyable	Main use
<code>std::unique_ptr</code>	Exclusive	No	Single owner heap object
<code>std::shared_ptr</code>	Shared	Yes	Multiple real owners
<code>std::weak_ptr</code>	Non-owning observer	Yes	Break <code>shared_ptr</code> cycles

Shallow Copy vs Deep Copy

Feature	Shallow Copy	Deep Copy
Copies pointer value	Yes	No, usually allocates new memory
Copies pointed data	No	Yes
Ownership risk	High	Lower
Performance	Faster	Slower
Common bug	Double delete, shared mutation	Expensive copy

Rule of 3 vs Rule of 5 vs Rule of 0

Rule	Meaning	When used
Rule of 3	Destructor, copy constructor, copy assignment	Pre-C++11 resource-owning classes

Rule	Meaning	When used
Rule of 5	Rule of 3 plus move constructor and move assignment	C++11+ manual resource management
Rule of 0	Write none of them manually	Modern C++ using RAII members

9. Common Mistakes

- Returning the address of a local stack variable.
- Forgetting `delete` after `new`.
- Using `delete` instead of `delete[]`.
- Mixing `malloc` with `delete` or `new` with `free`.
- Dereferencing a pointer after `delete`.
- Assuming `delete p` sets `p` to `nullptr`.
- Writing a destructor but forgetting copy constructor and copy assignment.
- Using `shared_ptr` when `unique_ptr` is enough.
- Creating `shared_ptr` cycles without `weak_ptr`.
- Manually managing memory when `std::vector`, `std::string`, or smart pointers would solve it.
- Confusing pointer lifetime with pointed-object lifetime.
- Thinking RAII is only for memory.

10. Edge Cases / Special Cases

- `delete nullptr;` is safe.
- `free(NULL);` is safe in C/C++.
- `delete` on a pointer not allocated by `new` is undefined behavior.
- `delete[]` must match `new[]`.
- `malloc(0)` behavior is implementation-defined; do not rely on it for meaningful storage.
- A pointer can be non-null but invalid.
- References can also dangle.
- A moved-from object is valid but its value may be unspecified.
- `std::make_unique<T>()` is preferred over `new T`.
- `std::make_shared<T>()` usually performs one allocation for the object and control block.
- `shared_ptr` reference counting is thread-safe for control block operations, but the pointed object is not automatically thread-safe.
- Destructors should generally be `noexcept`.
- If a base class is deleted through a base pointer, the base destructor should usually be virtual.

```

struct Base {
    virtual ~Base() = default;
};

struct Derived : Base {
    ~Derived() override = default;
};

```

```
Base* p = new Derived();  
delete p; // correct because Base destructor is virtual
```

11. How to Explain in Interview

"In C++, stack memory is automatically managed for local variables, while heap memory is used for dynamic lifetime objects. Manual allocation with `new/delete` or `malloc/free` is error-prone because it can cause leaks, dangling pointers, and double frees. Modern C++ prefers RAI: bind every resource to an object whose destructor releases it. For memory ownership, I use `unique_ptr` for exclusive ownership, `shared_ptr` only for real shared ownership, and `weak_ptr` to avoid cycles. If a class owns a raw resource, I follow the Rule of 3 or 5, but ideally I design with the Rule of 0 using standard containers and smart pointers."

12. Quick Revision Notes

Key Definitions

- **Stack:** Automatic memory for local variables and function calls.
- **Heap:** Dynamic memory used for flexible object lifetimes.
- **Leak:** Allocated memory is not released.
- **Dangling pointer:** Pointer to destroyed or freed memory.
- **RAII:** Resource cleanup through destructors.
- **Shallow copy:** Copies pointer/address only.
- **Deep copy:** Copies actual owned data into a new resource.
- **Rule of 3:** Destructor, copy constructor, copy assignment.
- **Rule of 5:** Rule of 3 plus move constructor and move assignment.
- **Rule of 0:** Prefer no custom special members by using RAI types.

Important Points

- `new` calls constructor; `delete` calls destructor.
- `malloc` does not call constructors; `free` does not call destructors.
- Do not mix allocation and deallocation families.
- Prefer `std::make_unique` and `std::make_shared`.
- Use `std::vector` instead of manual dynamic arrays.
- Use `std::lock_guard` for mutex RAI.
- Use `std::fstream` RAI for file handling.

Common Comparisons

- Stack vs heap: automatic lifetime vs dynamic lifetime.
- `new/delete` vs `malloc/free`: C++ object lifetime vs raw memory.
- `unique_ptr` vs `shared_ptr`: single owner vs shared owners.
- Shallow vs deep copy: address copy vs data copy.
- Rule of 3/5/0: manual ownership vs modern RAI design.

Must-Remember Facts

- `delete nullptr` is safe.

- `delete[]` must be used for `new[]`.
- Dereferencing dangling pointers is undefined behavior.
- Raw pointers should usually be non-owning in modern C++.
- `shared_ptr` cycles leak unless broken with `weak_ptr`.
- A class with raw owning pointer usually needs special member functions.

Interview Traps

- "Can I return a pointer to a local variable?" No.
- "Does `delete p` make `p` null?" No.
- "Is `malloc` enough for C++ objects?" No, constructors are not called.
- "Should I use `shared_ptr` everywhere?" No.
- "Does RAII handle exceptions?" Yes, destructors run during stack unwinding.

13. Practice Tasks

1. Write a function that leaks memory, then fix it using `delete`.
2. Rewrite the same function using `std::unique_ptr`.
3. Create a class with a raw `int*` and intentionally show the shallow copy problem.
4. Implement a deep copy constructor for that class.
5. Add copy assignment and handle self-assignment correctly.
6. Add move constructor and move assignment.
7. Replace the raw pointer class with `std::vector<int>` and observe how the Rule of 0 removes custom code.
8. Write a program that uses `std::shared_ptr` in a cycle, then fix it using `std::weak_ptr`.
9. Explain what happens line by line:

```
int* p = new int(10);
int* q = p;
delete p;
std::cout << *q << "\n";
```

10. Use AddressSanitizer or Valgrind on a small program with a leak or use-after-free.
11. Implement a simple RAII wrapper for a file or fake database connection.
12. Compare performance and clarity between a raw dynamic array and `std::vector<int>`.

14. Final Cheat Sheet

Core definition: C++ memory management is the control of object lifetime, ownership, allocation, and cleanup.

Why it matters: It prevents leaks, crashes, undefined behavior, security bugs, and performance problems.

Most asked questions:

- Stack vs heap?
- `new/delete` vs `malloc/free`?
- What is a memory leak?

- What is a dangling pointer?
- What is RAI?
- `unique_ptr` vs `shared_ptr`?
- Shallow copy vs deep copy?
- Rule of 3/5/0?

Common comparisons:

Comparison	One-line answer
Stack vs heap	Stack is automatic; heap is dynamically managed.
<code>new</code> vs <code>malloc</code>	<code>new</code> constructs objects; <code>malloc</code> only allocates raw memory.
<code>delete</code> vs <code>free</code>	<code>delete</code> destroys objects; <code>free</code> only releases raw memory.
<code>unique_ptr</code> vs <code>shared_ptr</code>	<code>unique_ptr</code> has one owner; <code>shared_ptr</code> has multiple owners.
Shallow vs deep copy	Shallow copies addresses; deep copies owned data.
Rule of 3 vs 5 vs 0	Manual copy cleanup, move-aware cleanup, or no manual cleanup.

One-line interview answer: Modern C++ memory management is about clear ownership and automatic cleanup: prefer RAI, containers, and smart pointers; use manual `new/delete` only when necessary and match every allocation with the correct deallocation.